Regression and Classification with a Neural Network
FYS-STK4155 - Project 2

Esme Hookham, Lukas Monrad-Krohn, and Andrea Rosendahl
University of Oslo
(Dated: November 9, 2026)

Selecting the right neural network configuration for any task can be challenging. In this project we investigate feed-forward neural networks for regression and classification. Neural networks learn their own features through hidden layers, offering a more flexible alternative to manually constructed polynomial features. We implement our own neural network code using backpropagation and investigate the effects of network architecture, activation functions, gradient-based optimisation and regularisation. We compare the regression results with ordinary least squares, Ridge and Lasso. For classification, we use the MNIST dataset of handwritten digits, applying the softmax activation function and cross-entropy cost to predict one of ten classes. We evaluate classification performance using accuracy and a confusion matrix, comparing our model with logistic regression and established implementations. The results show that for the Runge function, a neural network of three layers and 50 nodes with Adam stochastic gradient descent and ReLU activation with learning rate 0.005 and penalty $1e-4$ produces the lowest validation MSE using cross validation independent of regularisation. Lasso regression is the most efficient method, while the neural network is better at reproducing the edges of this function. A neural network with one hidden layer of 128 nodes, ReLU activation and L_2 regularisation achieves the highest classification accuracy of 0.9768 on the MNIST test set. Independent of the task, neural networks with the Adam optimiser and ReLU activation performed well in our tests. However, for less complex tasks regression methods can be the better choice.

I. INTRODUCTION

Neural networks involve interconnected layers of artificial neurons that transform input data to produce predictions. They efficiently represent complex and high-dimensional data and resolve non-linear aspects through hidden layers and non-linear activation functions. In this way, neural networks are crucial for solving problems that linear regression cannot solve. However, there are countless combinations of optimiser methods, regularisation norms and further hyperparameters and not all of them are suited for the same task. We aim to find the most suitable method for fitting the Runge function [1] and classify handwritten digits [2].

In Project 1 [3], we studied polynomial regression methods for approximating the Runge function, using ordinary least squares (OLS), Ridge and Lasso regression. These methods rely on manually constructed polynomial features, with model complexity controlled by the polynomial degree and regularisation. Although effective for relatively simple functions, this approach is unsuited for more complex or non-linear problems. Feed-forward neural networks (FFNN) offer a new method to approach similar problems by learning feature representations through successive layers of weighted transformations and activation functions which introduce depth, allowing the network to develop non-linear relationships. However, increasing the model complexity does not necessarily improve predictive performance and can lead to overfitting and higher computational costs.

First, we investigate which combination of activation functions, optimisation algorithms and regularisation allow a FFNN to approximate the Runge function [1], and evaluate its accuracy and applicability, by comparing the results with the best-performing OLS, Ridge and Lasso models from Project 1 [3].

The second aim of this project is to look at how neural networks approach multiclass classification using the full MNIST dataset of handwritten digits [2]. Unlike regression, which predicts continuous numerical values, classification assigns observations to discrete categories. We investigate how accurate a FFNN can correctly represent the digits through employing various hyperparameters in the setup. The results are compared with logistic regression and a confusion matrix is used to identify which digits are most frequently misclassified.

The report is structured as follows: Section II presents the methods for our analysis, while our results are described and discussed in section III. Lastly, we conclude in section IV. The code is available at `https://github.com/lukasMK01/FYS-STK4155/`.

II. METHODS

A. Cost functions and activation functions

Activation functions allow a NN to learn non-linear relationships and are specific to different problems. The sigmoid function and its derivative is

$$\sigma(z) = \frac{1}{1+e^{-z}}, \quad \sigma'(z) = \sigma(z)[1-\sigma(z)], \tag{1}$$

where the derivative is expressed with the function value. The derivative never exceeds 1/4, which can be a major issue in deep networks, as the gradient becomes smaller for each layer [4]. The average of the sigmoid function is close to 1/2, meaning the output of this function that is applied to the next layer is not centered, causing a slower learning. This function can saturate for large $|z|$, causing the derivative to be close to zero.

ReLU stands for the rectified linear unit, and the function and its derivative is

$$ReLU(z) = max(0,z), \quad ReLU'(z) = \begin{cases} 1 & z > 0, \\ 0 & z < 0, \end{cases} \tag{2}$$

which has the advantage that it does not saturate and the gradient stays intact for positive values. At the origin, the derivative is undefined, but in practice it is set at zero. A disadvantage with this function is the possibility of a dying ReLU, i.e., if all input values are negative then the output will be solely zeros. This results in a zero gradient and kills the rest of the training. Such a case can for instance happen with a large learning rate [4]. One way to fix this, is by employing the leaky ReLU

$$LReLU(z) = \begin{cases} z & z > 0, \\ \alpha z & z \leq 0, \end{cases} \alpha \approx 0.01, \tag{3}$$

causing the gradient to never entirely vanish by adding a small α term.

The identity function

$$I(z) = z, \quad I'(z) = 1, \tag{4}$$

is also included in this analysis when studying regression problems.

The softmax activation function

$$e(z) = \frac{exp(z - max(z))}{\Sigma exp(z - max(z))}, \tag{5}$$

is well suited for classification problems. This is a soft classifier, which means that it provides the probability of a dataset, in addition to a confidence of the classification.

A cost function is applied to the output to evaluate the neural network with the target at each step. The mean squared error (MSE) is often used for regression problems and we pair it with the identity function for activation in the output layer. The MSE and its derivative is given by:

$$MSE = \frac{1}{n}\Sigma_{i=0}^{n-1}(y_i - \tilde{y}_i)^2, \quad MSE' = \frac{2}{n}(y - \tilde{y}). \tag{6}$$

By adding the L1 and L2 norms to the MSE we get cost functions that penalise large weights and reduce the chance of overfitting as it is known from Ridge and Lasso regression. The resulting cost functions and its derivatives are

$$C_1 = \text{MSE} + \lambda \sum_i |W_i|, \qquad C_2 = \text{MSE} + \lambda \sum_i W_i^2, \tag{7}$$

$$\frac{\partial C_1}{\partial W_i} = \frac{\partial\, \text{MSE}}{\partial W_i} + \lambda\ \text{sign}(W_i), \qquad \frac{\partial C_2}{\partial W_i} = \frac{\partial\, \text{MSE}}{\partial W_i} + 2\lambda W_i. \tag{8}$$

The penalty contributes to the weight gradients only, but not to the bias gradients and we use only the pure MSE when evaluating test datasets with different methods to ensure comparability.

For classification into several mutually exclusive classes, the softmax activation function is paired with categorical cross-entropy:

$$C_{\text{CE}} = -\frac{1}{n}\sum_{i=1}^{n}\sum_{k=1}^{K} y_{ik}\log(p_{ik}). \tag{9}$$

where K is the number of classes, y_{ik} is the one-hot encoded target and p_{ik} is the predicted probability of class k. When softmax is combined with cross-entropy, the derivative with respect to the output logits simplifies to $p_{ik} - y_{ik}$. This allows the output error to be calculated directly during backpropagation, without explicitly computing the full derivative of softmax. The resulting cross-entropy cost is convex for a linear logistic regression model, but not generally for a neural network with hidden layers [4].

B. Setup of the FFNN

We implement a FFNN, which includes a backpropagation algorithm and a feed-forward algorithm. The feed-forward algorithm stores information from the layers, which is included in the backpropagation algorithm. Each layer receives an input, creates a weighted sum and a bias, and passes this through a nonlinear activation function to the next layer. This type of neural network is called a multilayer perceptron [4].

First, the Runge dataset is split into 80% training data and 20% test data, and is scaled by the training data. The magnitude of the initialised weights are important, as too large (small) weights can result in saturation (killing the signal). Therefore, the weights are initialised as a normal distribution, with a zero mean and the reciprocal of the length of the input layer as variance. The biases are initialised as small random numbers of order 10^{-9}, to prevent killing the training if all values are initialised at zero and the ReLU activation function is used in the first hidden layer.

The design matrix is chosen as the x values, while the number of hidden layers and nodes in each layer is flexible. The output activation function is the identity function with its derivative (Eq. 4), and the cost function is the MSE. To find the optimal hyperparameters in the setup of the FFNN, we use 5-fold cross validation (CV) [3, 4].

We use stochastic gradient descent (SGD) [5] with mini-batches of 15, unless specified otherwise. The SGD is both cheaper than full-batch gradient descent and includes noise to the training. Several optimiser steps in the gradient descent method from Project 1 are included, and in this analysis we focus on the plain, Adam [6] and RMSProp [7] gradient descent methods. We compare the results of the FFNN with the closed-form OLS linear regression problem from Project 1, using a 5th order polynomial, as that was our suggested best model for the Runge function.

C. Classification of the MNIST data

The MNIST dataset consists of 60,000 training images and 10,000 test images of handwritten digits from 0 to 9. Each image is flattened into a 784-dimensional vector, with pixel values scaled to the interval [0,1]. The training data is further divided into 80% fitting data and 20% validation data, using stratification to preserve the distribution of the digit classes. The validation data is used when comparing different model configurations, while the test data is reserved for the final evaluation.

The general FFNN structure and optimiser methods from the previous sections are retained, with the backpropagation algorithm adapted to calculate the gradients of the cross-entropy cost function. The target labels are converted into one-hot encoded vectors, allowing the network to compare its predicted probabilities against the correct digit class. A separate training function is implemented to record the training and validation costs, as well as the validation accuracy after each epoch. Accuracy is calculated by assigning each image to the digit with the highest predicted probability. We measure the quality of the classifier by calculating the accuracy, precision, recall, F1 and support [4].

The algorithm is validated against JAX automatic differentiation [8]. All networks were initialised using the same random seed (2026).

III. RESULTS AND DISCUSSION

A. Testing parameters in the FFNN

Figure 1 shows the train and test MSE for two FFNNs: 1 hidden layer of 50 nodes (1x50) and two hidden layers of 100 nodes each (2x100). Both models use the sigmoid function as the activation function for the hidden layers,

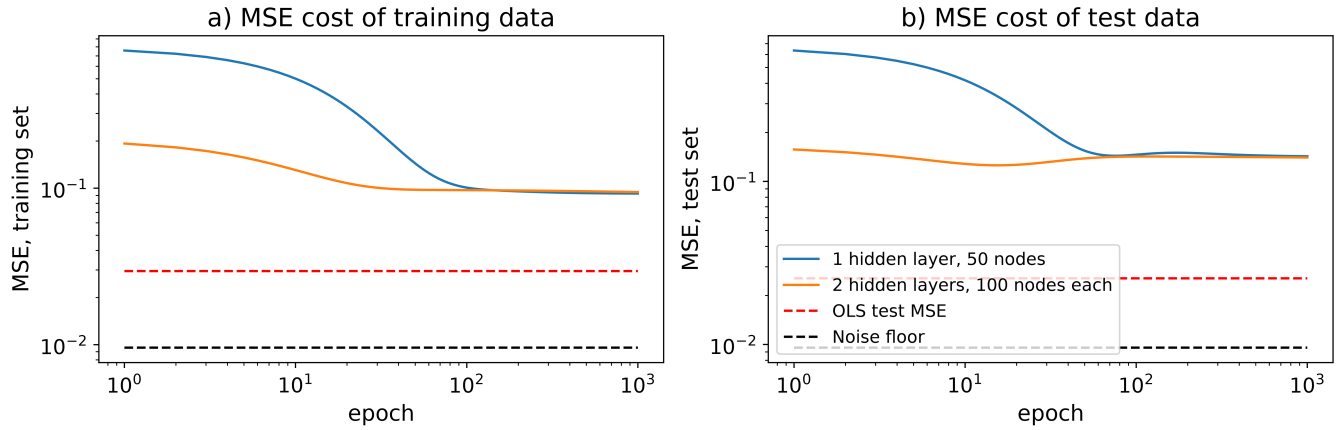


Figure 1: The MSE cost of the (a) training and (b) test data for two FFNNs, both using a sigmoid activation function in the hidden layer(s) and a full-batch plain gradient descent method with learning rate $\gamma = 0.001$. The models are compared to the MSE from the OLS closed-form solution (red) and the noise floor (black). The test data show an increase in MSE at 20 (100) epochs for the two-layer (one-layer) model, indicating overfitting.

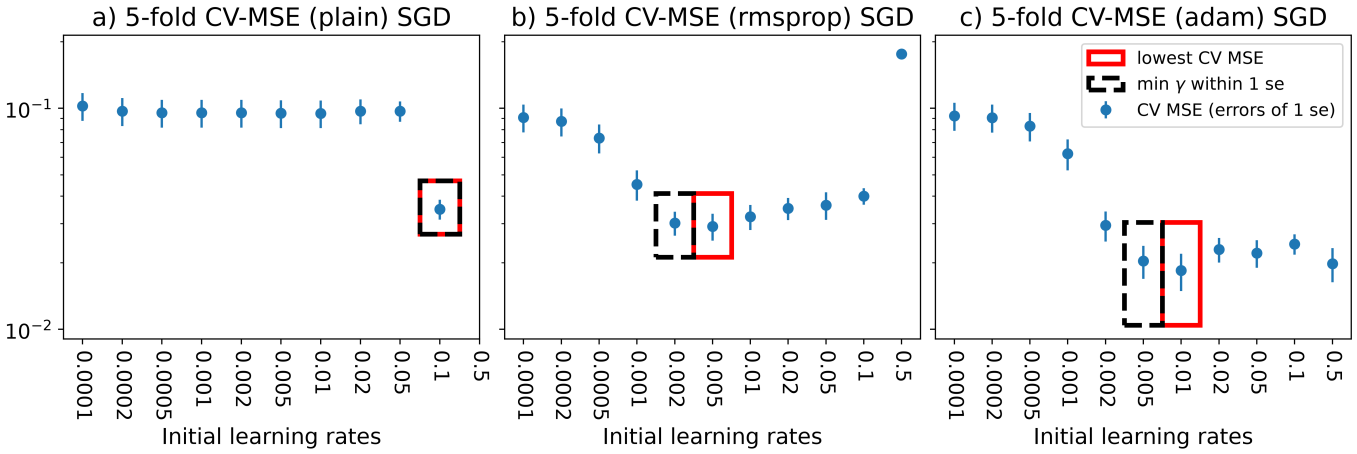


Figure 2: 5-fold CV MSE for the (a) plain, (b) RMSProp and (c) Adam SGD. Red box shows the learning rate that produced the lowest MSE, while the black box shows the lowest learning rate that has a MSE within one standard error (1 se) of the lowest MSE.

and the full-batch plain gradient descent method with learning rate $\gamma = 0.001$. The 1x50 model has a much higher initial MSE than the 2x100 model, but both reach a minimum MSE of 0.1 after 100 epochs in the training set. In the test set, the 1x50 (2x100) model's MSE increases slightly after around 100 (20) epochs, pointing to overfitting. This indicates that 1000 epochs is excessive and results in unnecessary computational costs. The final mean training (test) cost over the last 10 epochs is 0.092 (0.143) for the 1x50 model and 0.094 (0.140) for the 2x100 model. Both models have a higher cost than the OLS closed-form solution (red line), which has an MSE of 0.030 and 0.025 for the training and test data, respectively.

For further analysis, we choose the 1x50 settings, as the difference in minimum MSE is small between the models and one hidden layer is computationally cheaper than two hidden layers with twice the number of nodes (however, neither is expensive for such a dataset).

We compare the plain gradient descent with Adam and RMSProp, where the learning rate changes during training. We use SGD with mini-batches of 15 over 200 epochs for all three methods. To find the best initial learning rates, we use a 5-fold CV on the training data (Fig. 2). The lowest MSEs are found at initial learning rates $\gamma = 0.1$, $\gamma = 0.005$, and $\gamma = 0.01$ for the plain, RMSProp and Adam methods, respectively (red boxes). Following the one-standard-error rule, we choose the lowest learning rates which produce a MSE within one standard error of that with the lowest MSE. This means the optimal learning rates used for further analysis are $\gamma = 0.1$, $\gamma = 0.002$, and $\gamma = 0.005$ for the plain, RMSProp and Adam methods, respectively (black hatched boxes).

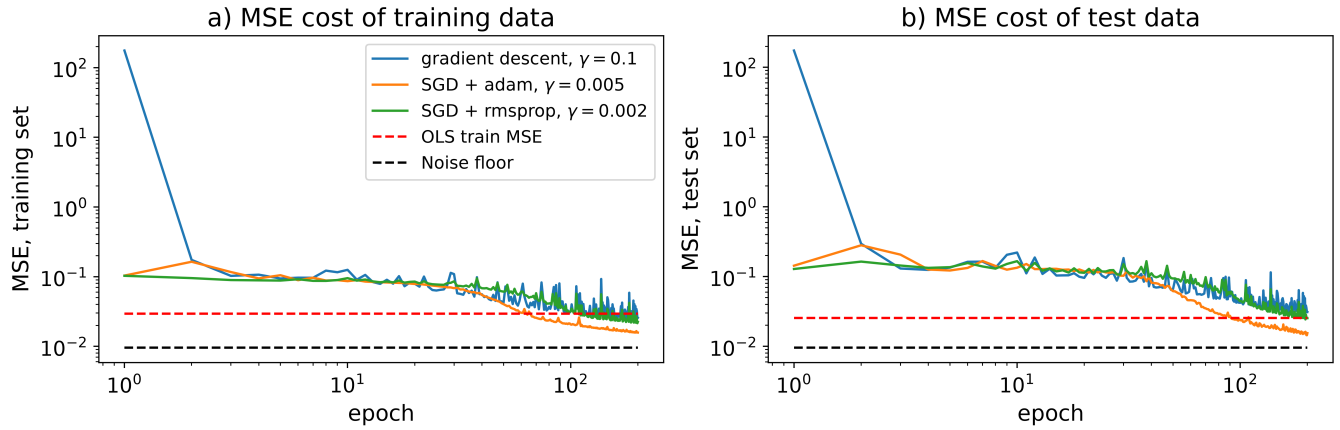


Figure 3: The (a) training and (b) test MSE using the plain, Adam and RMSProp SGD methods. The initial learning rates are chosen from the previous analysis. The models are compared to the OLS regression (red) and the noise floor (black)

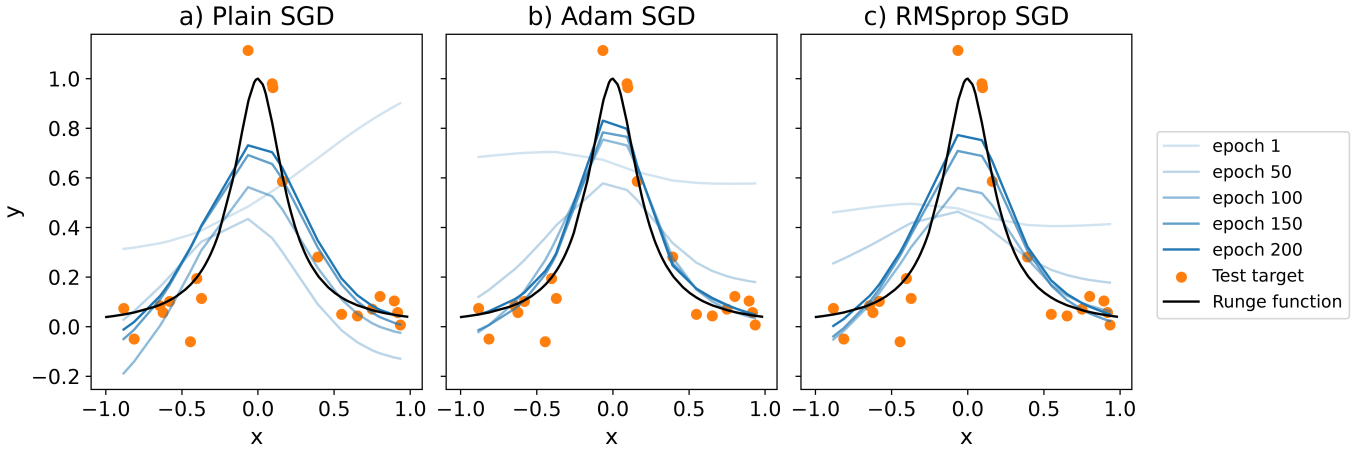


Figure 4: The predictions for the (a) plain, (b) Adam and (c) RMSProp SGD methods, for five selected epochs (ordered in colour). The predictions are compared to the test target points (orange) and the Runge function (black).

Fig. 3 shows the training and test MSE for the three SGD methods. All three SGDs are comparable to the OLS MSE and Adam has the lowest MSE of the last 10 epochs in both the training (0.016) and test (0.015) sets compared to the plain (0.032 and 0.039 for training and test) and RMSProp (0.025 and 0.028 for training and test) SGDs. The lower test MSE for Adam shows that this optimiser method does well on this particular test data.

To further evaluate the different SGDs, we compare the predicted output from the models with the Runge function during training (Fig. 3). The plain gradient descent method (a) reproduces the shape of the Runge function with one maximum, although the amplitude is clearly too weak. Both the Adam (b) and RMSProp (c) methods reproduce the function better at the last epoch, even though the maximum is too low. After 100 epochs the Adam prediction is nearly unchanged, while the plain and RMSProp are nearly unchanged after 150 epochs. This means that further training does not improve the models and the models have reached their optimal representation of the data. Only few points at the peak of the Runge function are included as targets. This scarcity of data points may make it more difficult for the neural network to reproduce the shape and amplitude of the function.

Based on this analysis, we choose the Adam SGD with initial learning rate $\gamma = 0.005$ for further analysis.

1. Analysis of activation functions

The previous analyses use the sigmoid function as the activation function of the hidden layer. The risks with relying on the sigmoid function include vanishing gradients (in the case of many numbers <1) and exploding gradients (many numbers >1). Though for a simple problem with 1-2 hidden layers, vanishing gradients have little effect [4], but

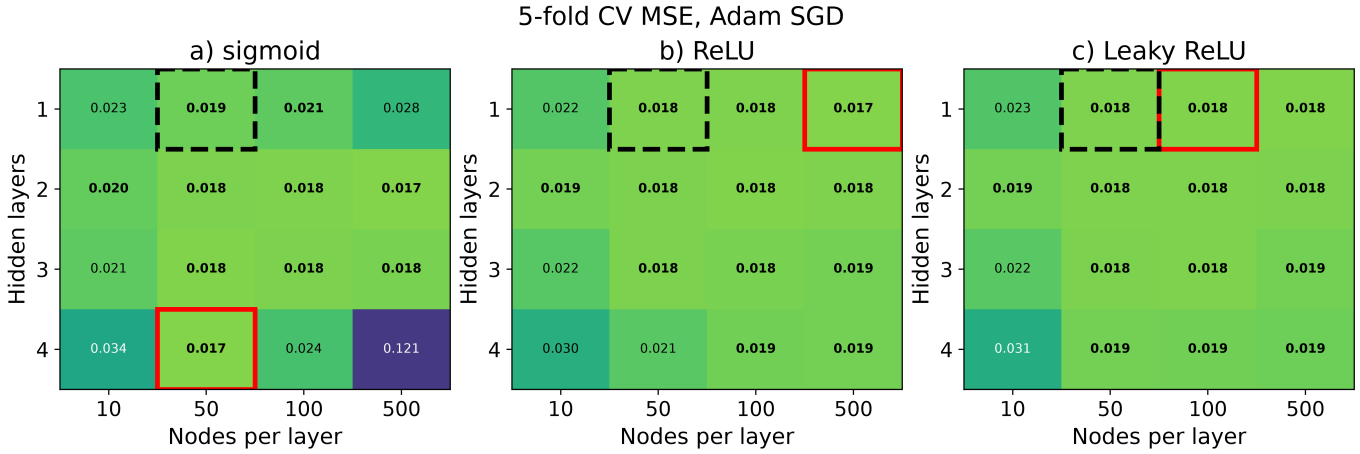


Figure 5: 5-fold CV MSE for various number of hidden layers and number of nodes per hidden layer using the (a) sigmoid, (b) ReLU and (c) Leaky ReLU activation functions as hidden layers. The red box shows the lowest CV MSE while the black box shows the "simplest" model that has an MSE within one standard error of the lowest CV MSE. Bold values are within one standard error.

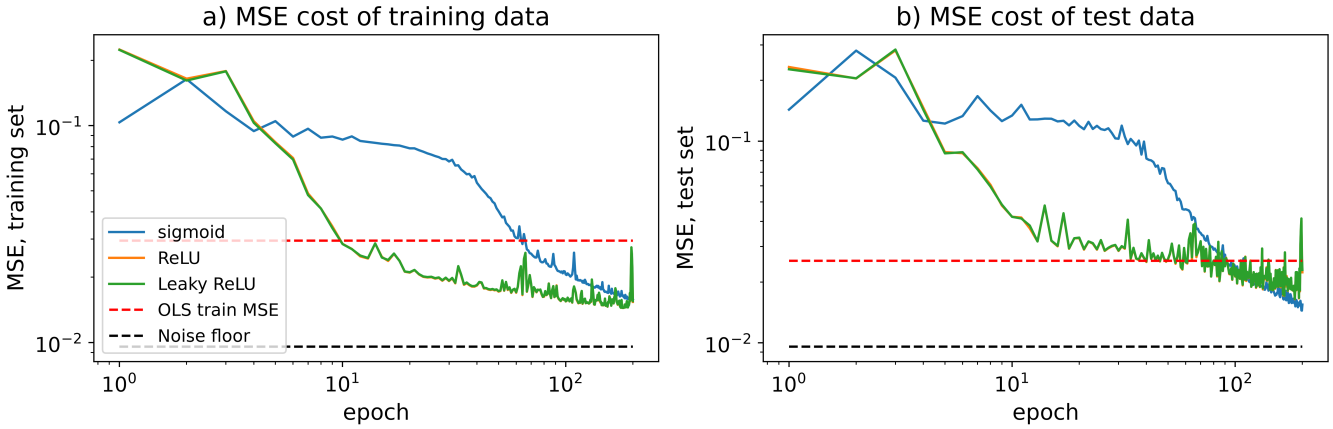


Figure 6: The MSE cost of the (a) training and (b) test data with models using the sigmoid, ReLU and the Leaky ReLU activation functions as hidden layers. The models are compared to OLS closed-form MSE and the noise floor.

in Fig. 3, the different methods show nearly constant MSE after a few epochs and do not decrease in MSE until around 20 epochs, which motivates concern. This highlights the complicated behaviour within activation methods, motivating an analysis between alternate functions. We therefore compare the sigmoid, the ReLU and the Leaky ReLU activation functions to find the best setup for the FFNN.

Fig. 5 shows the 5-fold CV MSE in dependence of number of hidden layers and nodes per hidden layer. The red boxes show the combination that produces the lowest CV MSE for the three models with different activation schemes. The sigmoid model yields the lowest MSE when using four hidden layers with 50 nodes per layer. In comparison, the ReLU and Leaky ReLU models yield the lowest MSE with a 1x500 and a 1x100 model, respectively, however most of the combinations tested are within one standard error of the lowest MSE. Following the one-standard-error rule, both the 2x10 model and a 1x50 model produce a CV MSE within one standard error of the lowest MSE with all three activation functions. Therefore, for consistency and choosing the simplest model within the MSE standard error, we choose a 1x50 setup for further analysis for all three models.

The ReLU and Leaky ReLU converge to low MSE at fewer epochs than the sigmoid, indicating that they are better models for the Runge problem (Fig. 6). All three models reach a lower test MSE than the OLS test MSE after 100 epochs. The mean MSEs over the last 10 epochs for the training (test) data are 0.016 (0.015), 0.019 (0.025) and 0.019 (0.026) when using the Sigmoid, ReLU and Leaky ReLU activation functions, respectively. The higher MSE for the ReLU and Leaky ReLU models are because of the sudden increase in MSE at the very end.However, this is just a spike and more epochs results in nearly constant MSE at the same level as before the peak (not shown).

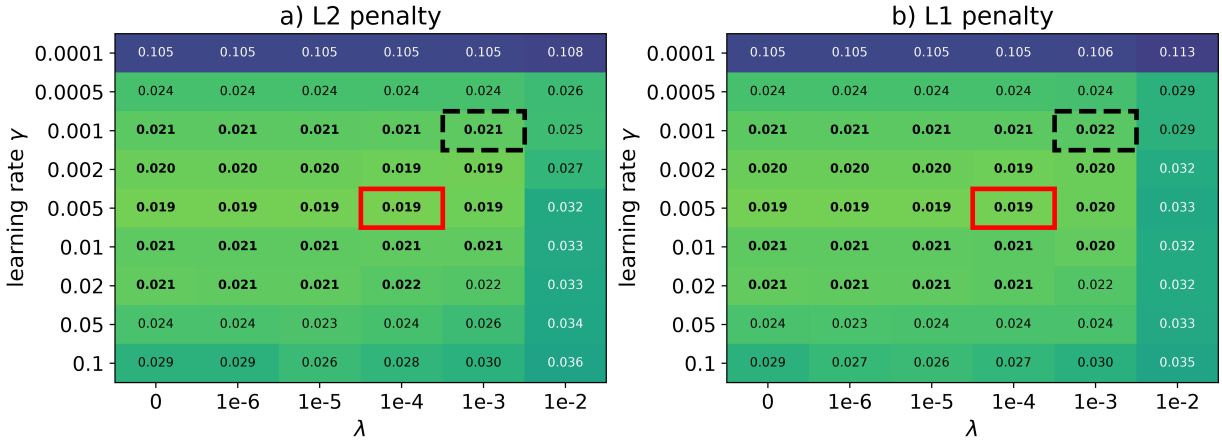


Figure 7: Heatmaps of 5-fold cross-validated MSE for the FFNN (Adam, one hidden layer with 50 nodes using ReLU activation function) with the a) L_2 and b) L_1 penalty for varying learning rates and λ. The red frame marks the minimum MSE, black dashed marks the optimal based on the one-standard-error rule and bold values are within one standard error.

The ReLU and Leaky ReLU are indistinguishable in the MSE cost. If the biases had initially been set with zeros, the training would have been killed with the ReLU function, and the Leaky ReLU function would prevent this by adding a small *alpha*. However, as the biases are initialised as small random numbers, this is not a problem, and may partly explain why the two models are so similar. Both models reproduce the Runge function reasonably well. After only 50 epochs, the ReLU and Leaky ReLU models show little improvement with training, suggesting that more epochs are redundant. The Leaky ReLU is generally a better activation function than the normal ReLU [4], however this does not show in our analysis as they are almost the same. This is likely because of the simple dataset with the Runge function, and may be more easily seen in deeper networks. Further, the ReLU is generally better than the sigmoid activation function, which is what we also see here, although the final test MSE is lower in the sigmoid model.

2. *Testing the L_2 and L_1 norms*

We apply the penalty parameter λ to the cost function used in the FFNN through the L_2 and L_1 norms as described in Eq. 7. To check the new implementation of the backpropagation algorithm we compare it to JAX. Both norms yield accurate results and agree with JAX to machine precision.

To find the best FFNN setup, we use k-fold CV to evaluate the FFNN at different learning rates between $1e-4$ and $1e-1$ and λ between $1e-6$ and $1e-2$ and 0. The cross-validated MSE is lowest at $\gamma = 0.005$ and $\lambda = 1e-4$ for both the L_2 and L_1 norms (Fig. 7 a,b). In general, both models produce very similar results with the best combinations between $\gamma = 0.001$ and 0.02 and $\lambda < 1e-3$. All hyperparameter combinations within this plateau lie within one standard error and according to the one-standard-error rule [4] the most penalised one at $\gamma = 0.001$ is the optimal learning rate. At learning rates smaller than 0.001 the FFNN converges too slowly to reach a good solution within 200 epochs and at learning rates above 0.02 it overshoots because the steps are on a similar order of magnitude as the weights themselves.

We continue with this optimal learning rate and test different architectures. Again, the L_2 and L_1 norms perform similarly everywhere but at the largest penalty (Fig. 8 a,b). This penalty is too large and leads to underfitting. Interestingly, the L_1 norm influences the deep models stronger. The 1x100 and 1x500 FFNNs are the only architectures where the penalty increases the accuracy (at $\lambda = 1e-4$ and $1e-3$), but they also performs worst of all architectures overall. This is inconsistent with the analysis of Fig. 5b, where 1x500 performed best and we do not see evidence for this difference in the code. The deep models generally perform better than those with a single hidden layer and a large width is only helpful for large penalties.

There can be no conclusion on whether the L_2 or L_1 norm is better suited for a fit of the Runge function based on CV, as L_2 performs marginally better on the widest model and L_1 does better at deep networks and the differences lie within one standard error. This means the feature selection mechanism that comes from setting some values to 0 in L_1 [9] is not important.

Before comparing the FFNN with the Ridge and Lasso regressions of project 1 we establish their benchmarks (Table I). The regression methods capture the strong peak of the Runge function well, but become unstable at the edges of the domain [3] (Fig. 9 a). As expected, none of the FFNNs show oscillations at the ends. We compare network

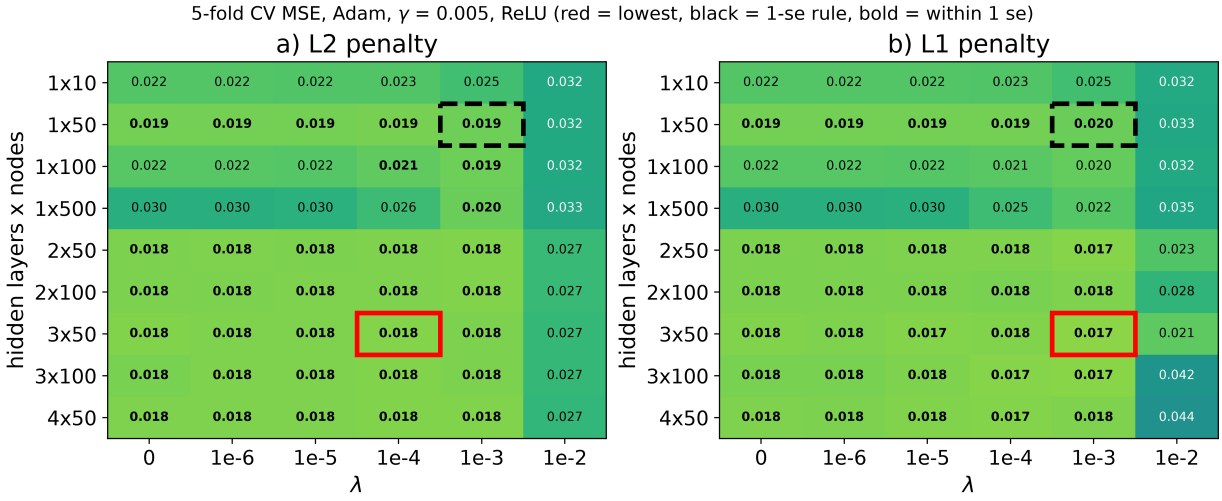


Figure 8: Heatmaps of 5-fold cross-validated MSE for the FFNN (Adam, one hidden layer with 50 nodes using ReLU activation function and $\gamma = 0.005$) with the a) L_2 and b) L_1 penalty for various architectures and λ. The red frame marks the minimum MSE, black dashed marks the optimal based on the one-standard-error rule and bold values are within one standard error.

configurations that match the parameters that yield the minimum MSE (red squares, marked *NN*) in Fig. 7 and 8 and the configurations with the optimal parameters based on the one-standard-error rule (black dashed rectangle, *NN 1x50*) from the same analysis. The minimum MSE model represents the peak of the Runge function best and is similar to the optimal parameter model (Fig. 9 a). However, when applied to the original test data set, the choice of hyperparameter makes a large difference. For any norm (none, L_2 or L_1) the FFNN with the hyperparameter combination that produces minimum CV MSE has at least 25% lower test MSE than the one according to the one-standard-error rule. The latter are also outperformed by each regression method with the corresponding norm. The one-standard-error rule is based on the fact that any improvement won by a more complex model is smaller than the uncertainty with which it can be quantified [4]. However, in this case applying this rule to select four hyperparameters (learning rate, penalty, model depth and width) makes a large difference. Based on the comparison of the test data MSE a FFNN with 3 hidden layers of 50 nodes, a learning rate of 0.005 and no penalty performs best.

These results only apply to this particular case based on the random train-test split, but they confirm the result of the 5-fold CV which found the same model to be best with low sensitivity to the penalty. Furthermore, the results for FFNN with small learning rates may become better if they are allowed to run for more than 200 epochs.

Table I: Using 5-fold CV to find the parameter combination and applying that to the entire test set (20%). The noise level of the test points is 0.0096.

Method	best CV-MSE	best polynomial degree	best λ	train MSE	test MSE
OLS	0.0266	8	–	0.0173	0.0187
Ridge	0.0266	12	$3.2e-6$	0.0156	0.0221
Lasso	0.0259	9	$1.0e-4$	0.0172	0.0184

B. Classification of the MNIST problem

We look at the MNIST dataset of handwritten digits [2] to further compare optimal hyperparameters, this time through classification. Starting with network architecture, we look at each hyperparameter and test for the most efficient value while keeping the remaining settings fixed. Throughout these experiments, cross-entropy is used as the cost function and accuracy as the main classification metric.

First we compare networks with different numbers of hidden layers and neurons. ReLU was used in the hidden layers and softmax in the output layer, with Adam, learning rate $\gamma = 0.001$, batch size 128 and 15 epochs kept fixed. The results show that increasing the width of a single hidden layer improves validation accuracy, from 0.9640 with 32 neurons to 0.9717 with 64 and 0.9751 with 128 neurons (Fig. 10). The best architecture tested was therefore [784, 128, 10]. However, adding a second hidden layer [784, 64, 32, 10] achieved a lower accuracy of 0.9712, suggesting

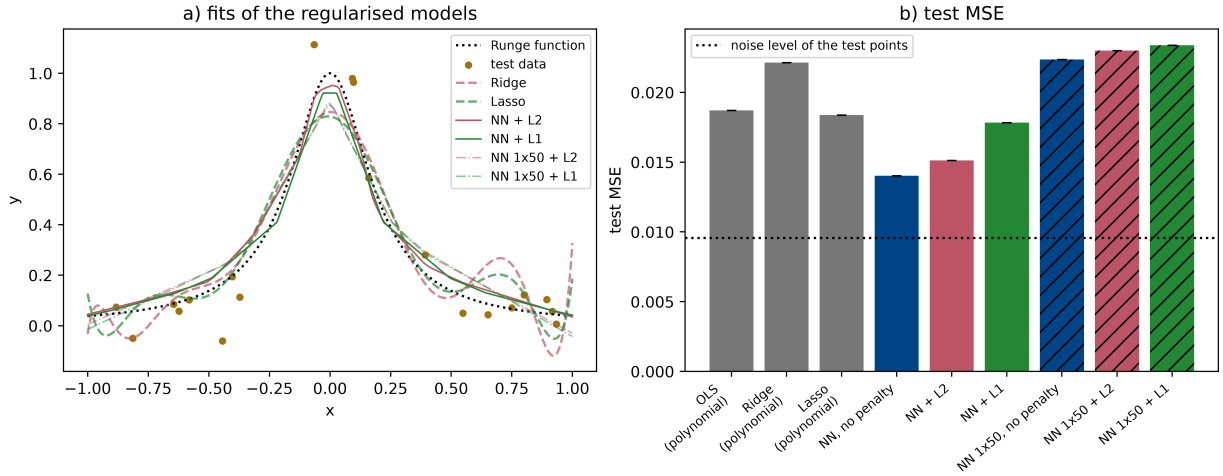


Figure 9: Comparison of polynomial regression and FFNNs with different norms for fitting the Runge function. a) shows the fits of each and b) shows the MSE of the test data compared to the noise level of the test data.

that additional depth did not improve performance under these conditions. The improvement from 32 to 128 neurons was small, at 0.0111, indicating diminishing returns from increasing network width under these conditions. However, these conclusions are conditional on the chosen learning rate, optimiser and activation functions, and a different combination of hyperparameters may produce different results.

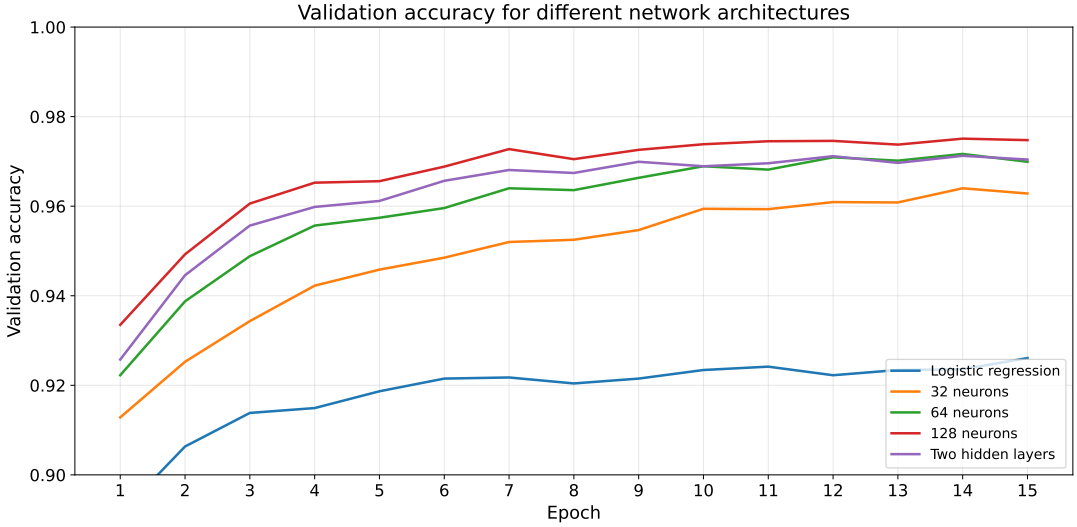


Figure 10: Validation accuracy for different network architectures.

Having selected [784, 128, 10], we compare ReLU, Leaky ReLU and sigmoid as hidden-layer activation functions (not shown), retaining softmax at the output. ReLU and Leaky ReLU produced almost identical results, with best validation accuracies of 0.9751 and 0.9750, and final validation costs of 0.0914 and 0.0938, respectively. Sigmoid performed less well, achieving a best validation accuracy of 0.9708 and final validation cost of 0.1026. This may reflect the smaller gradients associated with sigmoid, which can slow learning. ReLU was therefore retained, due to its slight advantage over Leaky ReLU and technically greater computational efficiency.

The learning rate was selected using five-fold CV (Fig. 11). Very small learning rates produced higher cross-entropy and lower accuracy, indicating insufficient progress over the fixed number of epochs. Performance improved as the learning rate increased, reaching a minimum mean cross-entropy of 0.09519 at $\gamma = 0.001$, with accuracy of 0.9718. Although $\gamma = 0.002$ achieved a marginally higher accuracy of 0.9721, its cross-entropy was higher at 0.10760. At larger learning rates, performance deteriorated substantially, with accuracy falling to 0.8427 at $\gamma = 0.1$ and 0.1049 at $\gamma = 0.5$. Using the one-standard-error rule, $\gamma = 0.001$ was selected for the remaining experiments.

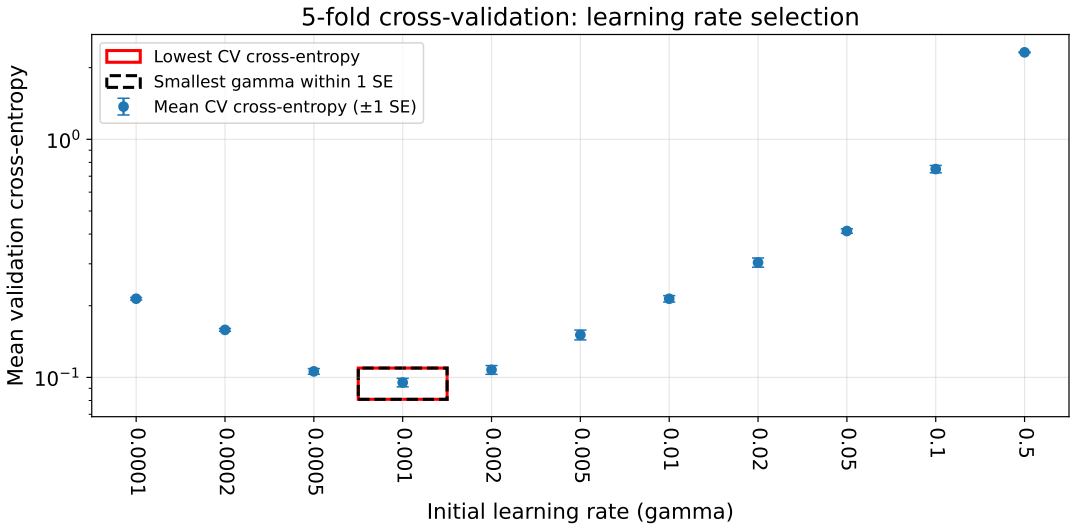


Figure 11: Five-fold CV results for learning-rate selection.

To investigate L_2 regularisation, the penalty parameter λ was varied between 0 and 0.1 (Fig. 12). A small amount of regularisation improved validation performance: without regularisation ($\lambda = 0$), the lowest validation cross-entropy was 0.0869, compared with 0.0836 at $\lambda = 0.0001$. The highest validation accuracy was 0.9752 at $\lambda = 0.00001$, compared with 0.9751 at both $\lambda = 0$ and $\lambda = 0.0001$. Thus, weak regularisation slightly improved cross-entropy, although accuracy remained almost unchanged.

Stronger regularisation reduced performance: the lowest validation cost increased to 0.1011 at $\lambda = 0.001$, 0.2635 at $\lambda = 0.01$, and 0.8172 at $\lambda = 0.1$. At the strongest penalty, maximum accuracy fell to 0.8520, indicating underfitting because the network was too restricted to learn the patterns in the images.

Without regularisation, training cost fell to 0.0158 by epoch 15, while validation cost was 0.0914, suggesting overfitting. At $\lambda = 0.0001$, training cost was higher (0.0267), but the lowest validation cost was lower (0.0836), consistent with improved generalisation. Overall, $\lambda = 0.0001$ was selected for its lowest validation cross-entropy, although the benefit was small. The results demonstrate that weak regularisation can help limit overfitting, while excessive regularisation causes underfitting.

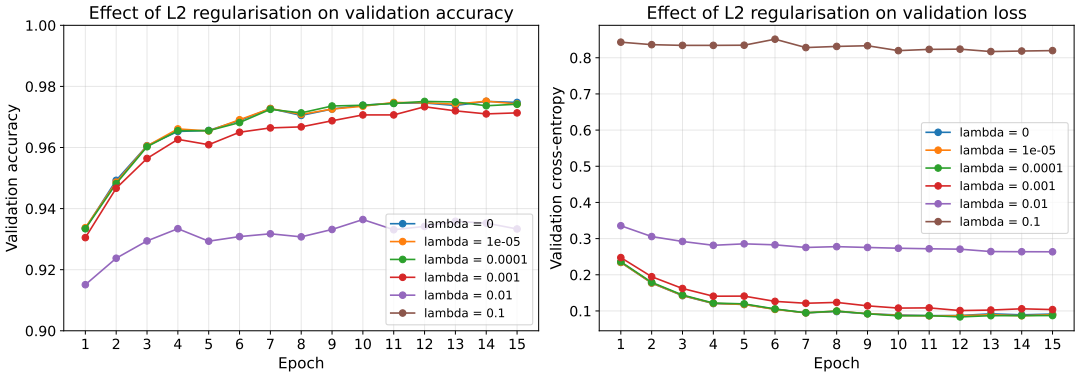


Figure 12: Effect of L2 regularisation on training and validation performance.

The selected FFNN achieved a test accuracy of 0.9768 and cross-entropy of 0.0751. The confusion matrix (Fig. 13) shows strong classification performance across all ten digits, with most predictions concentrated along the diagonal. The largest individual errors involved digits 3 misclassified as 5 and digits 9 misclassified as 4 (14 cases each). Digit 3 had the lowest recall (0.9574), while digit 8 had the lowest precision (0.9465).

Overall, the macro-average precision, recall and F1-score were 0.9766, 0.9769 and 0.9767, respectively. Their similarity indicates relatively consistent performance across classes. All digits achieved an F1-score above 0.96, demonstrating that the network generalises well across the different handwritten digits.

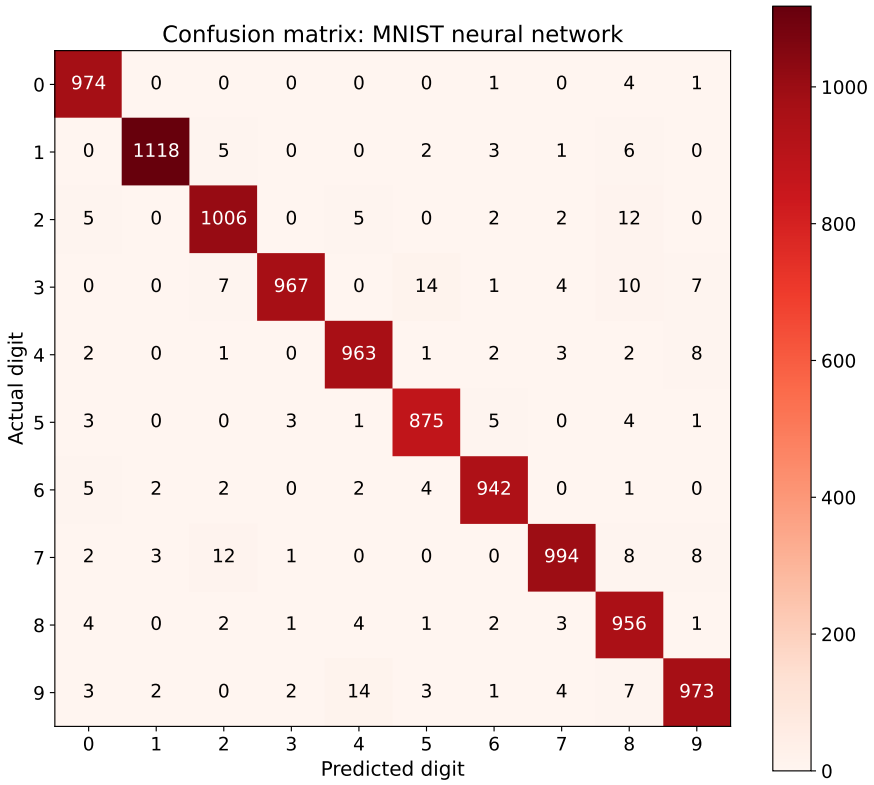


Figure 13: Confusion matrix for the final MNIST neural network.

Finally, to further evaluate our FFNN we compare it with logistic regression implemented both in our own code and using Scikit-Learn. Our logistic regression model achieves a test accuracy of 0.9265 and cross-entropy of 0.2659, compared to 0.9258 and 0.2715 for Scikit-Learn. The similarity between these results suggests that our implementation produces comparable performance to an established library. Both logistic regression models perform considerably worse than our FFNN, which achieves an accuracy of 0.9768 and cross-entropy of 0.0751. This improvement suggests that the hidden layer allows the network to learn more complex, non-linear relationships between pixel values, which are important for distinguishing handwritten digits. Notably, our FFNN achieves this performance despite being trained on 48,000 images, compared to the 60,000 used by Scikit-Learn. Overall, the results demonstrate the advantage of introducing a hidden layer for this classification task.

C. Comparing NNs with simpler regression methods

The analysis of the Runge function shows that there is little difference in MSE between the different FFNN configurations and linear regression. As the Runge function is a simple 1-dimensional dataset, linear regression works fine and can be computationally cheaper than using a neural network which is expensive to train. Further, a limitation with neural networks is that they are not interpretable, meaning they can only reproduce the data but not give an interpretation of what it means. As the Runge function only consists of 100 data points in this case, the test data only has 20 data points. This scarcity of data is a limitation on how well the FFNNs perform.

The optimisation of the FFNN depends on the chosen setup of the model, such as hidden layers, activation functions, learning rates and regularisation terms, and the randomness that is included. As seen in the analysis above, these choices were made based on CV of training data, and affect the test MSE results. The most suitable method to efficiently represent the Runge function is the Lasso regression. However, if accuracy at the domain edges outweighs computational efficiency and simplicity, a FFNN will be better suited.

A neural network with no hidden layers and a softmax output is equivalent to multinomial logistic regression [4]. This provides a useful baseline for evaluating the benefit of introducing hidden layers. In our MNIST experiments, our logistic regression model achieved a test accuracy of 0.9265 and cross-entropy of 0.2659, compared to 0.9768 and 0.0751 for our FFNN. This suggests that the additional flexibility provided by the hidden layer substantially improves

classification performance. Logistic regression makes predictions directly from the original pixel values, whereas the hidden layer allows the network to learn intermediate representations of the images. These may capture combinations of curves, edges and strokes that help distinguish between similar digits.

However, more complex models are not necessarily preferable for every problem. Linear and logistic regression are generally computationally cheaper, easier to interpret and require fewer hyperparameters. Neural networks require more computational resources and can be more prone to overfitting, particularly when training data is limited. Furthermore, their optimisation is non-convex, meaning that the final solution can depend on the initial weights and randomisation during training. Our earlier regression experiments also demonstrate that simpler models can perform well when the underlying relationship is sufficiently well represented by the chosen model. Therefore, the appropriate method depends on the complexity of the problem and the available data, rather than assuming that a neural network will always provide better results.

IV. CONCLUSION

In this report, we have studied FFNNs for both regression and classification analysis and explored various hyperparameters and choices for the setup of the FFNN.

Our regression analysis on the Runge function shows that the MSE from our FFNN is comparable to the MSE from simpler regression models. Through a 5-fold CV, we find that a 3x50 model with the ReLU function as the hidden layer and using Adam SGD with learning rate $\gamma = 0.005$ and penalty $\lambda = 1e - 4$ produces the optimal model. Our CV analysis shows that more complicated models produce lower MSEs, though through the one-standard-error rule we typically chose simpler models. However, the application of the one-standard-error rule to multiple parameters lead to the simpler, chosen optimal FFNN performed less well than one that only minimises MSE. Our results show that Lasso regression optimally reproduces the Runge function when considering computational efficiency. However, our FFNN reproduces the edges of this function more accurately.

Our results on the MNIST classification problem show that the FFNN produces a higher accuracy and lower cross-entropy than a logistic model. The hidden layer allows it to learn nonlinear relationships between pixel values and digit classes, whereas logistic regression is a linear classifier and therefore less accurate. From the 5-fold CV, we find that one hidden layer with ReLU activation, learning rate $\gamma = 0.001$ and penalty $\lambda = 0.0001$ produces the highest accuracy and lowest cross-entropy cost of 0.9768 and 0.0751, respectively. The classification performance is overall strong, as seen from the confusion matrix (Fig. 13), but digits 3 and 9 had the largest individual errors.

In CV analysis, the ranges of various hyperparameters are a choice that is limited by our expectations and time. Extending the ranges may result in other choices with lower MSEs or higher accuracy than the ones we found. Choices made from CV analysis affect the FFNN and therefore its cost. Considering that our optimal parameter values are not on the edges of these ranges, we believe that the chosen values are a good representation of the model when considering the cost. Furthermore, we only test certain combinations by testing parameters stepwise instead of the full parameter space.

The design matrix is chosen as only the x values. Alternatively, we can use x^2 or $|x|$ as the design matrix for a more simple curve to train the neural network against, however, as the original x provided sufficient results, we chose to keep this unchanged. In some cases, we found indication of overfitting, as the test MSE started to increase. For further work, we recommend including an early stopping metric, that can monitor the validation data and therefore stop the training when the cost starts to increase. We can also add gradient clipping, which will rescale the gradient to a selected threshold if the gradient exceeds this threshold.

[1] C. Runge, Zeitschrift für Mathematik und Physik **46**, 224 (1901).

[2] Y. LeCun, C. Cortes, and C. Burges, ATT Labs [Online]. Available: http://yann.lecun.com/exdb/mnist **2** (2010).

[3] L. R. A. Hookham, Esme; Monrad-Krohn, FYS-STK4155 - Project 1 (2026), URL `https://github.com/lukasMK01/FYS-STK4155/tree/main/project_1`.

[4] M. Hjorth-Jensen, *Applied data analysis and machine learning: Lecture notes*, `https://github.com/CompPhysics/MachineLearning` (2026).

[5] H. Robbins and S. Monro, The Annals of Mathematical Statistics **22**, 400 (1951).

[6] D. P. Kingma and J. Ba, in *International Conference on Learning Representations (ICLR)* (2015), arXiv:1412.6980.

[7] G. Hinton, *rmsprop: Divide the gradient by a running average of its recent magnitude* (2012), URL `https://www.cs.toronto.edu/~tijmen/csc321/slides/lecture_slides_lec6.pdf`.

[8] J. Bradbury, R. Frostig, P. Hawkins, M. J. Johnson, C. Leary, D. Maclaurin, G. Necula, A. Paszke, J. VanderPlas, S. Wanderman-Milne, et al., *JAX: composable transformations of Python+NumPy programs*, `http://github.com/google/jax` (2018).
[9] I. Goodfellow, Y. Bengio, and A. Courville, *Deep Learning* (MIT Press, Cambridge, MA, 2016).

Appendix A: Use of AI/LLM tools

A.1 Tools used: Claude (claude.ai, September 2026); GitHub Copilot (VS Code, throughout); UiO GPT (GLM5.3)
A.2 Text writing and editing.
No AI tools were used for text writing and editing.
A.3 Code generation and assistance.

Table II: AI use in coding (0=None, 1=Debugging, 2=Snippet, 3=Skeleton, 4=Substantial).

File & Code block	Level	Description
all	1	GitHub Copilot inline suggestions are a default setting and w
main.ipynb, task b-c)	1	help with visualisation of figures and solving bug with
README$_proj2.md$	4	fully AI generated readme file
main.ipynb	1	making sure all figures have a similar layout (only fontsizes, no
main.ipynb	2	sorting all functions into seperate .py files and ensuring no double definitions (
main.ipynb d)	1	removing unnecessary code and commer
main.ipynb d)	1	implementing the choice dict for the best s
`https://github.com/lukasMK01/FYS-STK4155/`	x	GitHub support